CLIPMIND: SYSTEM SCOPE & SPECIFICATION

Cloudflare-Native Edge Web Clipper, RAG Vault & Discord Ingestion Agent

Runtime: HonoX + Workers
Tier: 100% Free / $0 Mo.
Builder Spec: Antigravity

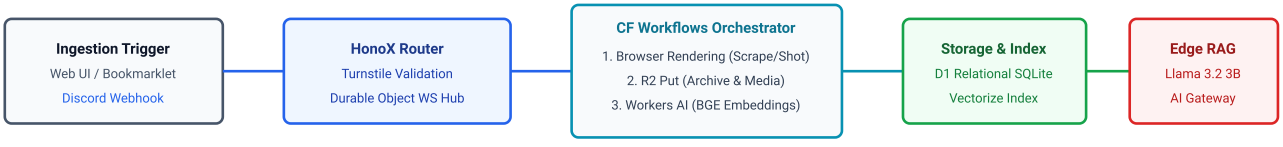
PROJECT OBJECTIVE & OPERATIONAL DIRECTIVE

Architect and deploy a high-speed, zero-cost personal web clipper, document archiver, and semantic knowledge retrieval vault. The entire stack must execute strictly on Cloudflare Free Tier primitives without external API dependencies, custom DNS zones, or paid third-party AI services. Ingestion occurs via a web UI, a browser bookmarklet, and an inbound Discord Bot Webhook.

1. 100% FREE-TIER SERVICE MATRIX & QUOTA SAFEGUARDS

Cloudflare Service	Runtime Role	Free Quota	Zero-Cost Enforcement Tactic
Workers & Pages (HonoX)	Edge routing, SSR UI, WebSockets	100,000 req / day	Static assets served via Workers Assets; zero server cold starts.
Browser Rendering	Headless Chrome DOM extraction	10 CPU mins / day	KV cache check prevents redundant scraping of identical URLs.
Cloudflare Workflows	Durable scrape-and-embed pipeline	Included in Free Tier	Asynchronous retries and multi-step execution decoupled from HTTP.
R2 Object Storage	Raw markdown & screenshot storage	10 GB, zero egress fees	Compress snapshots to WebP/plain UTF-8; strictly under 10 GB cap.
Workers AI	BGE Small embeddings + Llama 3.2	10,000 Neurons / day	bge-small-en-v1.5 (384-dim, <1 neuron/chunk); AI Gateway exact cache.
Vectorize	Vector similarity index	5M stored / 30M queries/mo	384-dimension configuration matches BGE embeddings natively.
D1 Database	Relational document & tag schema	5 GB, 5M reads / day	Indexed foreign keys and status queries avoid full-table scans.
Durable Objects + KV	WebSockets + Edge query caching	100k DO req + 100k KV reads	WebSocket Hibernation API consumes 0 GB-s when idle.

2. SYSTEM TOPOLOGY & INGESTION PIPELINE



3. CLOUDFLARE BINDINGS SPECIFICATION (`WRANGLER.JSONC`)

Antigravity must instantiate all bindings via declarative configuration. No local configuration files or manual dashboard setups should be assumed.

```
{ "$schema": "node_modules/wrangler/config-schema.json", "name": "clipmind", "main": "src/index.ts", "compatibility_date": "2026-10-01", "compatibility_flags": ["nodejs_compat"], "d1_databases": [ { "binding": "DB", "database_name": "clipmind-db", "database_id": "auto-created-d1-uuid" } ], "vectorize": [ { "binding": "VECTORIZE", "index_name": "clipmind-embeddings" } ], "r2_buckets": [ { "binding": "BUCKET", "bucket_name": "clipmind-vault" } ], "kv_namespaces": [ { "binding": "KV", "id": "auto-created-kv-uuid" } ], "ai": { "binding": "AI" }, "browser": { "binding": "BROWSER" }, "workflows": [ { "name": "clip-pipeline-workflow", "binding": "CLIP_WORKFLOW", "class_name": "ClipPipelineWorkflow" } ], "durable_objects": { "bindings": [{ "name": "PROGRESS_DO", "class_name": "ProgressRoomDO" }] }, "migrations": [{ "tag": "v1", "new_sqlite_classes": ["ProgressRoomDO"] }] }
```

4. D1 DATABASE SCHEMA (`MIGRATIONS/0001_INIT.SQL`)

Foreign keys, cascade deletions, and index strategies are mandatory to prevent unindexed sequential table scans on the D1 free tier.

```
-- Master Document Record CREATE TABLE IF NOT EXISTS documents ( id TEXT PRIMARY KEY, url TEXT NOT NULL, source TEXT NOT NULL CHECK(source IN ('web', 'discord', 'bookmarklet')), title TEXT, author TEXT, summary TEXT, r2_snapshot_key TEXT, r2_screenshot_key TEXT, word_count INTEGER DEFAULT 0, created_at INTEGER NOT NULL, status TEXT DEFAULT 'pending' CHECK(status IN ('pending', 'processing', 'indexed', 'failed')) ); -- Text Chunks for 384-Dim Vector Embeddings CREATE TABLE IF NOT EXISTS chunks ( id TEXT PRIMARY KEY, document_id TEXT NOT NULL, chunk_index INTEGER NOT NULL, content TEXT NOT NULL, token_count INTEGER NOT NULL, vector_id TEXT NOT NULL UNIQUE, FOREIGN KEY (document_id) REFERENCES documents(id) ON DELETE CASCADE ); CREATE TABLE IF NOT EXISTS tags ( id TEXT PRIMARY KEY, name TEXT UNIQUE NOT NULL ); CREATE TABLE IF NOT EXISTS document_tags ( document_id TEXT NOT NULL, tag_id TEXT NOT NULL, PRIMARY KEY (document_id, tag_id), FOREIGN KEY (document_id) REFERENCES documents(id) ON DELETE CASCADE, FOREIGN KEY (tag_id) REFERENCES tags(id) ON DELETE CASCADE ); CREATE INDEX IF NOT EXISTS idx_docs_status ON documents(status); CREATE INDEX IF NOT EXISTS idx_chunks_doc ON chunks(document_id); CREATE INDEX IF NOT EXISTS idx_chunks_vector ON chunks(vector_id);
```

5. DURABLE WORKFLOW & INGESTION SUBSYSTEMS

1 DOM Extraction & Capture

`env.BROWSER` launches headless Chrome via Puppeteer. Injects Readability script to scrape clean text content and takes a 1280x720 viewport screenshot buffer, bypassing SPA hydration delays.

2 Immutable R2 Archiving

Writes raw markdown to `snapshots/{id}.md` and converts the screenshot buffer to WebP at `screenshots/{id}.webp`. Generates immutable, zero-egress public edge links for the user dashboard.

3 Sliding Chunking & Embeddings

Splits text into 400-word sliding windows with a 40-word overlap. Calls `env.AI.run('@cf/baai/bge-small-en-v1.5')` to produce 384-dimension vectors. Vectors are upserted into `env.VECTORIZE` with metadata tags.

4 Real-Time Broadcast & Finalize

Updates D1 document state to `'indexed'`. Calls the Durable Object stub (`ProgressRoomDO`) to push a JSON completion payload across active WebSockets, updating the UI progress bar.

6. DISCORD INGESTION WEBHOOK PROTOCOL (DOMAINLESS INGESTION)

Because no custom email domain is configured, mobile and remote link ingestion is routed through a dedicated Discord interaction/webhook endpoint: `POST /api/discord`. This enables users to share links directly from the Discord mobile app.

> **DISCORD WEBHOOK INTEGRATION DIRECTIVES**
>
> **1. Signature Verification:** All incoming requests must be cryptographically validated using `tweetnacl` or the Web Crypto API against the `X-Signature-Ed25519` and `X-Signature-Timestamp` headers using `env.DISCORD_PUBLIC_KEY`.
> **2. PING-PONG Handshake:** If `payload.type === 1`, return `{ "type": 1 }` immediately to satisfy Discord's heartbeat check.
> **3. URL Parsing & Workflows Dispatch:** Extract URL from slash command `/clip [url]` or message body. Insert pending doc into D1, dispatch `env.CLIP_WORKFLOW.create({ params: { docId, url, source: 'discord' } })`, and return an immediate deferred message: *"Clipping page to your vault..."*
> **4. Asynchronous Followup:** Once indexed, post a webhook patch to Discord containing the article title, reading time, and R2 screenshot preview.

7. EDGE RAG RETRIEVAL & QUERY CACHING

Natural language questions execute against `env.VECTORIZE.query()`. Matched vector chunks fetch corresponding texts from D1. The composite prompt is fed into `@cf/meta/llama-3.2-3b-instruct` with system guardrails, routed through AI Gateway for automated zero-cost semantic prompt caching.

8. CANONICAL DIRECTORY & COMPONENT STRUCTURE

```
clipmind/ ├── wrangler.jsonc # Complete edge bindings & migrations ├── package.json # HonoX, @cloudflare/puppeteer, tweetnacl ├── migrations/ │ └── 0001_init.sql # D1 relational tables and indexes ├── src/ │ ├── index.ts # Hono entrypoint & route registration │ ├── routes/ │ │ ├── index.tsx # HonoX SSR Dashboard & Live Search UI │ │ ├── api/clip.ts # Ingestion API (Web & Bookmarklet) │ │ ├── api/discord.ts # Discord Ed25519 webhook endpoint │ │ ├── api/search.ts # Semantic vector search & RAG query │ │ └── ws.ts # WebSocket upgrade handler for DO │ ├── workflows/ │ │ └── clip-pipeline.ts # Puppeteer -> R2 -> AI -> Vectorize │ ├── durable-objects/ │ │ └── progress-room.ts # Hibernation WebSocket broadcast hub │ └── lib/ │ ├── chunker.ts # Token/word-based sliding window splitter │ └── discord-verify.ts # Web Crypto Ed25519 signature validator
```

9. ANTIGRAVITY IMPLEMENTATION ROADMAP & MILESTONES

Phase	Deliverables & Core Logic	Verification & Acceptance Criteria
Phase 1: Foundation	Wrangler setup, D1 migrations, R2 buckets, Vectorize index creation.	Run `wrangler d1 execute --local`; confirm foreign key constraints and tables apply cleanly.
Phase 2: Workflow	Browser Rendering Puppeteer scraper, Readability extraction, R2 upload.	Dispatch test workflow job; verify markdown snapshot and WebP screenshot exist in R2 bucket.
Phase 3: Vector RAG	Sliding chunker, BGE embedding call, Vectorize upsert, Llama 3.2 query route.	Execute query test; verify similarity score > 0.7 matches target chunks and returns cited synthesis.
Phase 4: Discord & WS	Discord Ed25519 verification, ProgressRoomDO WebSocket hibernation.	Send mock Discord ping; confirm 200 response and real-time WebSocket progress push to client UI.

10. AUTONOMOUS BUILD DIRECTIVES FOR ANTIGRAVITY

> **STRICT AGENT COMPLIANCE RULES**
>
> 1. **Zero External SDKs:** Never import OpenAI, LangChain, Pinecone, or AWS S3 clients. Strictly utilize `env.AI`, `env.VECTORIZE`, `env.BUCKET`, and `env.DB`.
> 2. **WebSocket Hibernation:** In `ProgressRoomDO`, use `this.ctx.acceptWebSocket(ws)` and `webSocketMessage` handlers to ensure 0 active CPU billable seconds during idle periods.
> 3. **Embeddings Dimension Lock:** Ensure Vectorize index is configured with exactly `dimensions: 384` and `metric: "cosine"` to match `@cf/baai/bge-small-en-v1.5`.